RecoverOS — 5-minute demo script

Razorpay AI Buildathon · Track 3 · recorded from /checkout and the dashboard

Green = say this, out loud *Amber italics = do this on screen* Plain = notes to yourself

Two tabs in one window: `/checkout` left, `/` (dashboard) right. Phone on speaker, on the desk. Warm every route once before recording. Twilio trial calls ask you to press a key first.

0:00 Cold open: the phone rings first

> *[Before you press record: on /checkout, pay ₹4,999 and press Failure on the mock bank page. Start recording the instant the timeline shows "Calling … via Twilio". Camera or screen on the phone; say nothing until it rings.]*

> *[Phone rings. Answer on speaker. Press a key at the Twilio prompt. Let the Hindi script play. After the question, answer in one sentence: "My card was blocked after I lost it."]*

> *[Cut to the dashboard tab. The event line reads what you just said.]*

> That call was placed by software, forty seconds after I failed a payment. It knew my renewal had failed, decided a call was worth more than a link, asked me why in my own language, and wrote my answer into an audit trail before I put the phone down.
>
> Nobody asked it to make that call. And in a minute I will show you the cases where it refuses to.

0:35 The number (dashboard)

> *[Cursor on the incremental ledger.]*

> This is RecoverOS, a revenue recovery agent for subscription merchants on Razorpay. Every product in this category reports one number: money recovered. Most of it was arriving anyway. Issuers retry on their own, customers pay late on their own, and the vendor bills for all of it.
>
> We report a smaller number. Incremental recovered, measured against a randomized holdout: five percent of eligible customers we deliberately leave alone. That is the figure on screen, with its confidence interval. Everything else today is in service of making that number honest.

1:00 The two refusals

> *[Point to the two cards side by side.]*

> Nobody else will show you a recovery agent refusing to act. We show it refusing for two different reasons, and we price both.
>
> On the left, the scorer's judgement: contacting this dormant subscriber would destroy more subscription value than it recovers, so we do nothing, and we book what we protected next to what we gave up. It could be wrong, and it is labelled as a judgement.
>
> On the right, the regulator. Not a judgement at all. It cites the code, names the rule, and prices the face value at stake. Face value, not lost revenue: an episode refused at ten at night is eligible again at nine in the morning.

[Click "Inspect a refused episode →". Panel scrolls into view.]

> The citation, the EIR arithmetic, and the append-only audit trail, per episode.

[If the amber banner is visible, say this now, once:]

> We are recording after nine PM, so the quiet-hours gate is switched off on this server and it says so on screen. Earlier tonight, with it on, it refused our own test call at a quarter to eleven. That refusal is in the queue.

1:40 How that call happened (checkout tab)

[Switch to /checkout. The timeline from the cold open is still on screen.]

> Here is how the call at the start happened. This is Razorpay Checkout in test mode against a real order. I was the customer, and I failed my own renewal.

[Point down the timeline rather than re-running it. If you want it live again, click Pay, Netbanking, Failure, and the phone rings a second time.]

> Razorpay reports the failure. We enrich it from Razorpay's payment API, sign it exactly as their webhook would be signed, and hand it to the production webhook route. From here nothing is demo-specific.

[Timeline fills. Read it as it lands.]

> Diagnosed as a bank decline. Scored. The policy approved a voice call, because at this amount, with a phone on file, a call has the highest expected incremental value net of cost and churn. That decision is deterministic. No model touched it.

> Twilio read a Hindi script the system wrote for this episode, asked why, and the answer you heard me give is the last line of the timeline, and the last line of the audit trail.

2:40 Close the loop

[Dashboard. Click "POST /api/webhooks/razorpay". Wait one second. Click "Customer pays the link".]

The other half of the loop. Razorpay tells us the link was paid. The row flips to recovered and the ledger moves.

[Click "Customer pays the link" again.]

Same event again: a duplicate, no second transition. Razorpay redelivers webhooks, so both halves of the loop have to be idempotent.

3:05 The kill switch

[Scroll to the degradation panel. Open the outage.]

Razorpay's brief leads with detecting degradation. This is a real detector, keyed by method, issuer and network, watching fifteen-minute windows against a smoothed baseline. I am feeding it a synthetic spike on one issuer; everything it does next is production code.

[Episodes move to HELD.]

Three times baseline, guards passed, window opened. New failures on that issuer are held instead of retried into a dead bank.

[Close the outage. Drain.]

Hysteresis close, then a jittered drain, and the held episodes go back through the policy gate as if they had just arrived.

3:45 The measurement

[Scroll to the benchmark panel. Click the toggle to "Churn ignored" BEFORE you speak.]

How do we know any of this is worth money? Twelve hidden worlds, three thousand episodes each, four strategies on identical worlds, paired per seed. The simulator imports nothing from our decision code.

This is the leaderboard every vendor would show you: recovered minus cost. A plain rules engine that messages everyone is first, at nineteen lakh twenty-one thousand a world. We are third, at eighteen lakh sixty-six. On this board, our product has no reason to exist.

[Click the toggle to "Churn priced". The cards re-rank; Rules drops to the bottom.]

> Same run. One change: we price the subscribers those messages drove away. Rules was hiding one lakh seventy-seven thousand of churn per world, because it contacted nine hundred and thirteen customers to our three hundred and fifty-six. It falls to last, seventy-nine thousand below doing nothing at all. We move ahead of silent retry by fourteen thousand a world, and the oracle, which reads the answer key, is only sixty thousand ahead of it. We capture about a quarter of what is available, and we can name most of what we miss.
>
> The full report, fifty thousand episodes across twenty worlds, says the same thing: we beat silent retry on twenty of twenty, and one lakh eighty-nine thousand of that survives the compliance gate armed inside the run.
>
> Three things we would rather not have to say. Drop the churn column and a plain rules engine wins, here and in the full report, and this product has no reason to exist; pricing churn is the whole claim. Most of our win is a blunt rule, not clever targeting: at the shipped rate the fatigue term simply forbids a second contact. And our own yardstick was wrong once, understating the ceiling by two and a half times, until a reviewer found it. Fixing it made us look worse.

4:40 Close

> *[Back to the dashboard header.]*

> The architecture in one line: a language model may propose, a deterministic policy disposes, and only the executor holds credentials. A model that proposes sending money gets a reject, and there is a test for it.
>
> Every number you saw regenerates with one command: npm run verify. If the committed report disagrees with what the code produces, the check fails. Thank you.

Cuts if you run long: drop the kill switch first, then the second click on "Customer pays the link".